

# XGBoost (Extreme Gradient Boosting)

## Complete Step-by-Step Mathematical Processing Notes

### What is XGBoost?

---

XGBoost (Extreme Gradient Boosting), introduced by Chen & Guestrin (2016), is an optimised and scalable implementation of gradient boosting. It extends vanilla Gradient Boosting with three key additions:

1. **Second-order Taylor expansion** of the loss (uses both gradient *and* Hessian).
2. **Regularised objective function** — adds  $L_1$  and  $L_2$  penalties on leaf weights to control overfitting.
3. **Exact & approximate split-finding algorithms** for scalable, parallelised tree construction.

### Notation Reference

---

| Symbol                 | Meaning                                                           |
|------------------------|-------------------------------------------------------------------|
| $n$                    | Number of training samples                                        |
| $x_i \in \mathbb{R}^d$ | Feature vector of sample $i$                                      |
| $y_i$                  | True label of sample $i$                                          |
| $\hat{y}_i^{(t)}$      | Prediction for sample $i$ after $t$ trees                         |
| $f_t(x)$               | The $t$ -th tree (weak learner)                                   |
| $T$                    | Number of leaves in a tree                                        |
| $w_j$                  | Weight (score) of leaf $j$                                        |
| $q(x)$                 | Function mapping sample $x$ to its leaf index                     |
| $I_j$                  | Set of sample indices in leaf $j$ : $I_j = \{i \mid q(x_i) = j\}$ |
| $g_i$                  | First-order gradient of loss w.r.t. $\hat{y}_i$                   |
| $h_i$                  | Second-order gradient (Hessian) of loss w.r.t. $\hat{y}_i$        |
| $\lambda$              | $L_2$ regularisation coefficient on leaf weights                  |
| $\alpha$               | $L_1$ regularisation coefficient on leaf weights                  |
| $\gamma$               | Minimum loss reduction required to make a split                   |

### Step 1 — The Additive Model

---

XGBoost builds an ensemble of  $M$  trees additively:

$$\hat{y}_i^{(0)} = \text{base score (e.g., 0.5 or mean of } y)$$

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

where  $f_t$  is the new tree added at step  $t$ .

After all  $M$  trees, the final prediction is:

$$\hat{y}_i = \hat{y}_i^{(0)} + \sum_{t=1}^M f_t(x_i)$$

**Key Insight:** At each step  $t$ , we are learning the single best tree  $f_t$  to add to the current ensemble. We do **not** retrain previous trees.

## Step 2 — The Objective Function

At step  $t$ , we want to find the tree  $f_t$  that minimises the following **regularised objective**:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

where  $\Omega(f_t)$  is the regularisation term penalising tree complexity:

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 + \alpha \sum_{j=1}^T |w_j|$$

- $\gamma T$  penalises the **number of leaves** (tree complexity).
- $\frac{1}{2} \lambda \sum w_j^2$  is  $L_2$  regularisation on leaf weights (prevents large leaf scores).
- $\alpha \sum |w_j|$  is  $L_1$  regularisation (encourages sparsity in leaf weights).

## Step 3 — Second-Order Taylor Expansion of the Loss

The loss  $L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i))$  is difficult to optimise directly over all possible tree structures. XGBoost approximates it using a **second-order Taylor expansion** around  $\hat{y}_i^{(t-1)}$ :

$$L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) \approx L(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2$$

where the gradient  $g_i$  and Hessian  $h_i$  are:

$$g_i = \frac{\partial L(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}} \quad h_i = \frac{\partial^2 L(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2}$$

Summing over all  $n$  samples and dropping the constant term  $L(y_i, \hat{y}_i^{(t-1)})$  (since it does not depend on  $f_t$ ), the objective simplifies to:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^n \left[ g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t)$$

**Why second-order?** Vanilla Gradient Boosting uses only  $g_i$  (first order). Using both  $g_i$  and  $h_i$  gives a more accurate local approximation of the loss surface, enabling better step sizes and faster convergence.

## Step 4 — Re-expressing the Objective in Terms of Leaves

A regression tree  $f_t$  maps each sample  $x_i$  to a leaf  $q(x_i) \in \{1, \dots, T\}$  and assigns it the leaf weight  $w_{q(x_i)}$ , so  $f_t(x_i) = w_{q(x_i)}$ .

We can therefore re-group the sum *by leaf* instead of by sample:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{j=1}^T \left[ \left( \sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left( \sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T + \alpha \sum_{j=1}^T |w_j|$$

For notational convenience, define per-leaf sums:

$$G_j = \sum_{i \in I_j} g_i \quad H_j = \sum_{i \in I_j} h_i$$

Substituting:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{j=1}^T \left[ G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma T + \alpha \sum_{j=1}^T |w_j|$$

## Step 5 — Solving for the Optimal Leaf Weights

For a **fixed tree structure**  $q$ , the objective is a sum of independent quadratics in each  $w_j$ . Ignoring  $L_1$  for clarity (setting  $\alpha = 0$ ), each term is:

$$\tilde{\mathcal{L}}_j = G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2$$

This is a convex quadratic in  $w_j$ . Setting the derivative to zero:

$$\frac{\partial \tilde{\mathcal{L}}_j}{\partial w_j} = G_j + (H_j + \lambda) w_j = 0$$

Solving:

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

Substituting  $w_j^*$  back into the objective gives the **optimal tree-structure score** (a lower score means a better tree):

$$\tilde{\mathcal{L}}^{(t)}(q) = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T$$

**Interpretation:**  $w_j^*$  is the optimal prediction for leaf  $j$ . The term  $\frac{G_j^2}{H_j + \lambda}$  acts as a *score* for how well leaf  $j$  reduces the loss. A larger magnitude means a more informative split. The  $\lambda$  in the denominator prevents any single leaf from dominating

(regularisation).

## Step 6 — The Split-Finding Algorithm (Gain Criterion)

We cannot enumerate all possible tree structures. XGBoost uses a **greedy, level-by-level split search**. For a candidate split that divides a node's sample set  $I$  into a left child  $I_L$  and right child  $I_R$  (where  $I = I_L \cup I_R$ ), define:

$$G_L = \sum_{i \in I_L} g_i, \quad H_L = \sum_{i \in I_L} h_i, \quad G_R = \sum_{i \in I_R} g_i, \quad H_R = \sum_{i \in I_R} h_i$$

The **Gain** from making this split is the reduction in the objective:

$$\text{Gain} = \frac{1}{2} \left[ \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

### Interpretation of Each Term

| Term                                        | Meaning                           |
|---------------------------------------------|-----------------------------------|
| $\frac{G_L^2}{H_L + \lambda}$               | Score of left child after split   |
| $\frac{G_R^2}{H_R + \lambda}$               | Score of right child after split  |
| $\frac{(G_L + G_R)^2}{H_L + H_R + \lambda}$ | Score of parent node before split |
| $\gamma$                                    | Penalty for adding one more leaf  |

**Pruning Rule:** If  $\text{Gain} \leq 0$ , the split does **not** improve the objective (after accounting for the complexity penalty  $\gamma$ ), and the node is **not split** (pre-pruning). This is XGBoost's built-in regularised pruning — no separate post-pruning pass is needed.

### Algorithm: Exact Greedy Split Finding

For each node, iterate over all features and all possible split points:

**Step 1.** Sort samples in the node by feature value  $x^{(k)}$  for each feature  $k$ .

**Step 2.** Scan from left to right, accumulating  $G_L$ ,  $H_L$  and computing  $G_R = G - G_L$ ,  $H_R = H - H_L$  (where  $G = \sum_{i \in I} g_i$ ,  $H = \sum_{i \in I} h_i$ ).

**Step 3.** Compute Gain at each split point.

**Step 4.** Choose the feature and split point with the **maximum Gain**.

**Step 5.** If  $\max(\text{Gain}) > 0$ , perform the split; otherwise, make this node a leaf.

## Step 7 — Computing $g_i$ and $h_i$ for Common Loss Functions

---

### MSE Loss (Regression)

$$L = \frac{1}{2}(y_i - \hat{y}_i)^2 \quad \Rightarrow \quad g_i = \hat{y}_i - y_i \quad , \quad h_i = 1$$

### Log-Loss / Binary Cross-Entropy (Binary Classification)

Let  $p_i = \sigma(\hat{y}_i) = \frac{1}{1 + e^{-\hat{y}_i}}$  be the predicted probability. Then:

$$L = -[y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

$$g_i = p_i - y_i \quad h_i = p_i(1 - p_i)$$

### Huber / MAE Loss (Robust Regression)

For Huber loss with threshold  $\delta$ :

$$g_i = \begin{cases} \hat{y}_i - y_i & \text{if } |\hat{y}_i - y_i| \leq \delta \\ \delta \cdot \text{sign}(\hat{y}_i - y_i) & \text{otherwise} \end{cases} \quad h_i = \begin{cases} 1 & \text{if } |\hat{y}_i - y_i| \leq \delta \\ 0 & \text{otherwise} \end{cases}$$

**Key Insight:**  $h_i$  acts as a **confidence weight** for each sample. Samples with higher Hessian values contribute more to the leaf weight computation. This is why XGBoost handles imbalanced data well via the `scale_pos_weight` parameter.

## Step 8 — Model Update

---

After the optimal tree  $f_t$  with structure  $q^*$  and leaf weights  $\{w_j^*\}$  is found, the ensemble is updated:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta \cdot f_t(x_i)$$

where  $\eta$  (learning rate / shrinkage) scales down the contribution of each new tree. A smaller  $\eta$  requires more trees but gives better generalisation.

## Step 9 — Complete Training Algorithm

---

1. Initialise  $\hat{y}_i^{(0)}$  (e.g.,  $\hat{y}_i^{(0)} = 0.5$  or mean of  $y$ ).
2. **For**  $t = 1$  **to**  $M$ :

(a) Compute per-sample gradients and Hessians:

$$g_i = \partial_{\hat{y}_i^{(t-1)}} L(y_i, \hat{y}_i^{(t-1)}) \quad h_i = \partial_{\hat{y}_i^{(t-1)}}^2 L(y_i, \hat{y}_i^{(t-1)})$$

(b) Build tree  $f_t$  by greedily finding splits that maximise Gain:

$$\text{Gain} = \frac{1}{2} \left[ \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

(c) Assign optimal weights to each leaf  $j$ :

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

(d) Update predictions:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta \cdot w_{q(x_i)}^*$$

**3. Output:** Final model  $\hat{y}_i = \hat{y}_i^{(0)} + \eta \sum_{t=1}^M f_t(x_i)$

## Step 10 — Worked Numerical Example (Regression)

---

**Dataset:** Three samples, MSE loss.

| $i$ | $x_i$ | $y_i$ | $\hat{y}_i^{(0)}$ |
|-----|-------|-------|-------------------|
| 1   | 1     | 3     | 2.0               |
| 2   | 2     | 5     | 2.0               |
| 3   | 3     | 4     | 2.0               |

**Initial prediction:**  $\hat{y}^{(0)} = \bar{y} = (3 + 5 + 4)/3 = 4.0$  (correcting the table above for illustration — let us use  $\hat{y}^{(0)} = 4$ ).

**Compute  $g_i$  and  $h_i$  (MSE:  $g_i = \hat{y}_i - y_i$ ,  $h_i = 1$ ):**

$$g_1 = 4 - 3 = 1, \quad g_2 = 4 - 5 = -1, \quad g_3 = 4 - 4 = 0$$

$$h_1 = h_2 = h_3 = 1$$

**Try split:**  $x \leq 1.5$  (left:  $\{1\}$ , right:  $\{2, 3\}$ ), with  $\lambda = 1$ ,  $\gamma = 0$ :

$$G_L = g_1 = 1, \quad H_L = 1 \quad G_R = g_2 + g_3 = -1, \quad H_R = 2$$

$$\text{Gain} = \frac{1}{2} \left[ \frac{1^2}{1+1} + \frac{(-1)^2}{2+1} - \frac{(1-1)^2}{3+1} \right] - 0 = \frac{1}{2} \left[ \frac{1}{2} + \frac{1}{3} - 0 \right] = \frac{1}{2} \times \frac{5}{6} = 0.4167$$

**Optimal leaf weights:**

$$w_L^* = -\frac{G_L}{H_L + \lambda} = -\frac{1}{1+1} = -0.5$$

$$w_R^* = -\frac{G_R}{H_R + \lambda} = -\frac{-1}{2+1} = +0.333$$

Update (with  $\eta = 0.3$ ):

$$\hat{y}_1^{(1)} = 4.0 + 0.3 \times (-0.5) = 3.85$$

$$\hat{y}_2^{(1)} = 4.0 + 0.3 \times (0.333) = 4.10$$

$$\hat{y}_3^{(1)} = 4.0 + 0.3 \times (0.333) = 4.10$$

These updated predictions are then used to compute new  $g_i, h_i$  for the next tree  $f_2$ , and the process repeats.

## Key Hyperparameters

| Parameter        | Role                                                              |
|------------------|-------------------------------------------------------------------|
| n_estimators     | Number of trees $M$ .                                             |
| learning_rate    | $\eta$ : shrinks each tree's contribution.                        |
| max_depth        | Maximum depth of each tree (controls model capacity).             |
| lambda           | $L_2$ regularisation on leaf weights (default 1).                 |
| alpha            | $L_1$ regularisation on leaf weights (default 0).                 |
| gamma            | Minimum gain to allow a split (pre-pruning threshold).            |
| subsample        | Fraction of training rows sampled per tree.                       |
| colsample_bytree | Fraction of features sampled per tree.                            |
| min_child_weight | Minimum sum of $h_i$ in a leaf ( $= H_j \geq \text{threshold}$ ). |

**min\_child\_weight** corresponds directly to the Hessian sum: a leaf is only created if  $H_j \geq \text{min\_child\_weight}$ . This prevents splits on very few or uncertain samples.

## XGBoost vs. Vanilla Gradient Boosting — Mathematical Summary

| Aspect              | Gradient Boosting          | XGBoost                                   |
|---------------------|----------------------------|-------------------------------------------|
| Taylor expansion    | First-order only ( $g_i$ ) | Second-order ( $g_i$ and $h_i$ )          |
| Leaf weight formula | Mean of residuals in leaf  | $w_j^* = -G_j / (H_j + \lambda)$          |
| Split criterion     | Variance reduction         | Gain = $\frac{1}{2}[\dots] - \gamma$      |
| Regularisation      | None (tree depth only)     | $L_1, L_2, \gamma$ on objective           |
| Pruning             | Post-pruning               | Pre-pruning via $\gamma$ (Gain $\leq 0$ ) |
| Parallelism         | None (sequential splits)   | Feature-level parallel split search       |

## One-Line Definition

**XGBoost** builds an additive ensemble of trees by, at each step, fitting the **second-order Taylor approximation** of the regularised loss, deriving **closed-form opti-**

**mal leaf weights**  $w_j^* = -G_j/(H_j + \lambda)$  and a **Gain criterion** to greedily select splits  
— combining speed, regularisation, and mathematical rigour into one framework.